DESHMUKH TECHNOLOGIES

DTTP · V1.6

Trainee Program

Deshmukh Technologies Trainee Program

A 90-day full-stack software development and professional development program that prepares trainees to build, test, debug, explain, document, and maintain production software.

PRIMARY OUTCOME
Junior Full-Stack Developer

FRONTEND
React + TypeScript

BACKEND TRACKS
Java / Spring Boot or Python / FastAPI

DATABASE
MySQL / PostgreSQL

DELIVERY
Git + GitHub + Docker + CI/CD + AWS

DURATION
90 Days · 12 Phases

Learn. Build. Solve. Review. Deploy. Grow.

INTERNAL TALENT DEVELOPMENT STANDARD

DOCUMENT	VERSION	CLASSIFICATION	OWNER
DTTP Program Handbook	1.6	Internal Training Standard	Deshmukh Technologies

HANDBOOK

Contents

PART I — PROGRAM FOUNDATION

01 Program Purpose
02 Program Vision
03 Program Mission
04 Development Model
05 Technology Architecture
06 Trainee Journey
07 90-Day Master Roadmap
08 Training Methodology
09 Daily Development Cycle

PART II — TECHNICAL CURRICULUM

10 Phase 1 — Orientation
11 Baseline Assessment
12 Git & GitHub
13 Phase 2 — Programming
14 Java Track
15 Spring Boot
16 Python Track
17 FastAPI
18 Frontend
19 Database
20 REST API

PART III — PROJECT & ENGINEERING

21 Major Training Project
22 Full-Stack Architecture
23 Authentication & Security
24 Testing
25 Debugging
26 Docker
27 CI/CD
28 AWS
29 Project Management

PART IV — ASSESSMENT & CAREER

30 Definition of Done
31 Mentor System
32 Daily Report
33 Weekly Review
34 Competency Model
35 Performance Status
36 Weekly Score
37 Final Assessment
38 Graduation Levels
39 Production Readiness Gate
40 Career Progression
41 DTTP Management System
42 Final DTTP Standard

PART I Program Foundation

01 Program Purpose

The Deshmukh Technologies Trainee Program (DTTP) is a structured professional development program designed to transform freshers, interns, graduates, and entry-level candidates into capable software engineers.

The program combines technical education, practical coding, full-stack development, project-based learning, engineering practices, mentorship, code review, testing, debugging, security, DevOps, professional development, and continuous assessment.

> The program does not define success as completion of 90 days. Success means the trainee can demonstrate the ability to build, test, debug, explain, document, and maintain software.

WHO THE PROGRAM IS FOR

- Freshers and recent graduates
- Interns converting into a structured path
- Career-switchers with basic programming exposure
- Entry-level hires who need a company standard

COMPANION DOCUMENTS

- This handbook — 90-day path and graduation
- Software Setup Guide — Windows station and Day-1 sign-off
- Self-Growth Program — ten-level knowledge map
- Corporate Profile — identity, values, and method

The program is a supervised professional environment with attendance, reviews, Git history, and a production-readiness gate.

02 Program Vision

> Build skilled, responsible and production-ready software professionals who can understand business requirements, develop complete applications, solve problems independently and contribute effectively to real Deshmukh Technologies projects.

03 Program Mission

1. Establish strong programming fundamentals.
2. Develop modern full-stack development skills.
3. Teach professional software engineering practices.
4. Provide practical project experience.
5. Develop independent problem-solving ability.
6. Build communication and teamwork skills.
7. Introduce testing, security and deployment.
8. Measure capability continuously.
9. Provide a structured path from trainee to developer.
10. Create a repeatable internal talent-development system.

04 Three-Dimensional Development Model

Every trainee is evaluated across three dimensions. A trainee must develop all three.

A. Technical

- Programming
- Frontend
- Backend
- Database
- APIs
- Testing
- Security
- Cloud

B. Engineering

- Git
- Architecture
- Clean code
- Debugging
- Code review
- CI/CD
- Docker
- Documentation

C. Professional

- Communication
- Discipline
- Teamwork
- Ownership
- Time management
- Accountability
- Problem solving

05 Technology Architecture

COMMON FOUNDATION

PROGRAMMING | OOP | DATA STRUCTURES | ALGORITHMS | WEB | SQL | GIT / GITHUB | REST APIS

FRONTEND

HTML | CSS | JAVASCRIPT | TYPESCRIPT | REACT

ENGINEERING

TESTING | SECURITY | DOCKER | CI/CD | AWS | MONITORING

BACKEND — TRACK A

JAVA | SPRING BOOT | SPRING DATA JPA | HIBERNATE

BACKEND — TRACK B

PYTHON | FASTAPI | SQLALCHEMY

06 Trainee Journey

01 Application	02 Screening	03 Technical Assessment
04 Interview	05 Selection	06 Orientation
07 Baseline Assessment	08 Foundation	09 Frontend
10 Backend Specialization	11 Database & APIs	12 Full-Stack Project
13 Testing	14 Security	15 Docker
16 CI/CD	17 AWS	18 Final Project
19 Final Assessment	20 Production Readiness	21 Junior Full-Stack Developer

07 90-Day Master Roadmap

Phase	Days	Outcome
1	1–5	Professional and engineering foundation
2	6–15	Programming and problem solving
3	16–25	Web and frontend foundation
4	26–35	React + TypeScript
5	36–45	Java / Spring Boot or Python / FastAPI
6	46–55	Database + REST API
7	56–65	Full-stack project
8	66–72	Authentication + security
9	73–78	Testing + debugging
10	79–82	Docker
11	83–86	CI/CD + AWS
12	87–90	Final project + assessment

PHASE DELIVERABLES

Phase	The trainee should produce
1	Baseline profile, professional standards acknowledgment, first Git repo
2	Solved problem set with commits and explanations
3–4	Responsive React pages for login, dashboard, and employee list
5–6	Working REST API and SQL schema for employees and departments
7	End-to-end create/list employee flow
8	Authenticated roles and protected routes
9	Automated tests and a written debug note for one real bug
10–11	Docker Compose runbook and a CI workflow
12	Final demo, documentation, and production-readiness evidence

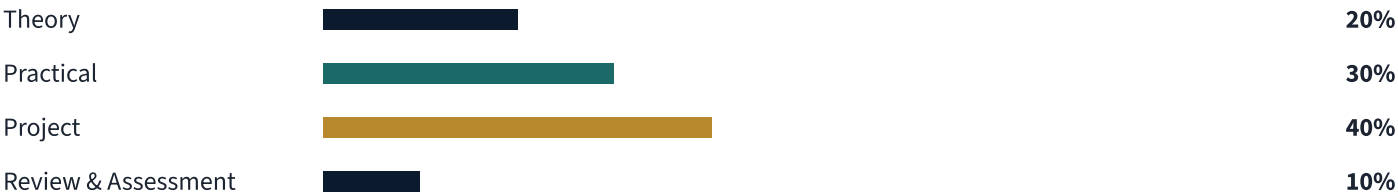
WEEK-BY-WEEK OPERATING PLAN

Use this when a mentor asks “what should this trainee be doing right now?”

Days	Trainee work	Mentor looks for
1–5	Read the three official documents. Complete baseline. Open the first repo.	Can summarize how the company works. First daily report exists.
6–10	Language drills: types, functions, collections, errors.	Commits show small problems with input and output.
11–15	Beginner then intermediate problems. One oral walkthrough.	Can dry-run a loop and name the failing case.
16–25	HTML/CSS then JavaScript form, DOM table, fetch.	No React yet. Keyboard use. Fetch error state.
26–35	React layout, routes, login, protected employees, mocked service.	API calls are not inside JSX.
36–45	Backend folders, health, login, employee create/list.	400/401/409 are distinct. One service test.
46–55	Schema, joins, attendance, leave apply.	Join written without the ORM.
56–65	Wire React to the real API. Dashboard, profile, departments.	Network tab matches the contract.
66–72	JWT, roles, CORS, injection notes.	EMPLOYEE cannot create or approve.
73–78	Leave tests, one written bug, one measured query.	Failing test first. Two timings recorded.
79–86	Compose, Actions, AWS service-purpose notes.	CI fails on a broken test. Health endpoint works.
87–90	Stories, README, demo, oral review, readiness gate.	Trainee explains without reading slides.

08 Training Methodology

Recommended allocation of trainee time:



> The trainee should spend more time building software than listening to someone explain software.

09 Daily Development Cycle

LEARN | UNDERSTAND | PRACTICE | IMPLEMENT | TEST | DEBUG | COMMIT | PULL REQUEST | CODE REVIEW | IMPROVE

This should become the trainee's normal development habit.

PART II Technical Curriculum

10 Phase 1 — Orientation

Days 1–5. Orientation establishes company context and professional standards before technical training begins.

COMPANY

- Deshmukh Technologies
- Vision and mission
- Products and services
- Projects and teams
- Development methodology
- Customer expectations

PROFESSIONAL STANDARDS

- Attendance and working hours
- Communication and meetings
- Workplace conduct
- Confidentiality
- Information security
- Responsibility

ORIENTATION READING

Before Day 6, the trainee reads and can summarize the Corporate Master Profile, this handbook (purpose, daily cycle, Definition of Done), the Self-Growth Program, and the **Trainee Software Setup & Environment Verification** guide. Day 1 environment work follows that guide. Project work does not start until the mentor marks the environment Pass or Conditional.

Day	Focus	Written output
1	Company, values, and software environment sign-off	Setup evidence pack + one-page company summary
2	Professional standards and security	Signed acknowledgment + first daily report
3	Baseline assessment	Baseline Skill Profile
4	GitHub access and first repository	Empty project README and first commit
5	Mentor meeting	Week-1 plan with one measurable objective

11 Baseline Assessment

Before technical training begins, assess the trainee and record a **Baseline Skill Profile**.

PROGRAMMING

- Logic
- Variables
- Conditions
- Loops
- Functions

TECHNICAL

- Java
- Python
- JavaScript
- SQL
- Git

PROBLEM SOLVING

- Logical reasoning
- Coding problem

COMMUNICATION

- Self-introduction
- Technical explanation
- Problem explanation

HOW TO RECORD THE BASELINE

Score each area L0–L5. Write one sentence of evidence, not a slogan. The baseline is a starting point, not a grade.

Area	Level	Evidence
JavaScript	L2	Built a form and fetch demo with mentor help
SQL	L1	Can explain SELECT/WHERE, has not written a join
Git	L1	Has committed locally, has not opened a pull request

12 Git & GitHub

Git is mandatory. Full Windows install, SSH, and Day-1 verification are in the **Trainee Software Setup** guide. This section is the Git workflow used after the machine is verified. Trainees must learn repository, clone, commit, branch, push, pull, merge, rebase, stash, revert, pull request, code review, and conflict resolution.

STANDARD WORKFLOW

TASK BRANCH CODE TEST COMMIT PUSH PULL REQUEST REVIEW FIX APPROVE MERGE

NAMING

```
Branch   dttp/<initials>/leave-approve-api
Commit   feat(leave): add approve endpoint and role check
```

Useful prefixes: `feat`, `fix`, `test`, `docs`, `chore`. One purpose per commit.

PULL REQUEST BODY

1. **What** changed
2. **Why** it changed
3. **How** it was tested
4. **What** a reviewer should look at first
5. **Risks** or follow-up work

A pull request with no test note is not ready for review.

```
## What
Add leave approve API for HR and ADMIN.

## Why
HR cannot currently decide a pending request.

## Test
- unit: employee cannot approve
- API: 401 / 403 / 409
- manual: HR approves one pending leave

## Review first
LeaveService.approve() role check

## Risk
Already-decided leaves must stay unchanged
```

REVIEW COMMENTS THAT HELP

Weak comment	Useful comment
“This is wrong.”	“Duplicate email should be `409` from the service, not `500` from the controller.”
“Clean this up.”	“Move `fetch('/api/employees')` into `employeeService.ts`.”
“Add tests.”	“Add `employee_cannot_approve` before merging.”
“Looks good.”	“Approve path is clear. Please add the already-decided `409` case.”

13 Phase 2 — Programming

Days 6–15.

Variables · Data types · Operators · Conditions · Loops · Functions · Collections · Strings · Exceptions · OOP · Data structures · Algorithms

BEGINNER EXERCISES

- Reverse string
- Palindrome
- Fibonacci
- Factorial
- Prime
- Largest element

INTERMEDIATE EXERCISES

- Two Sum
- Anagram
- Missing number
- Duplicate detection
- Frequency counting
- Sorting, searching, LinkedList

Every problem commit includes the prompt, one example input, the output, and the failing case. “Solved Two Sum” with no example is not accepted.

Problem	Example	Expected
Reverse string	`DTTP`	`PTTD`
Palindrome	`level`	`true`
Two Sum	`[2, 7, 11, 15]`, target `9`	`[0, 1]`
Duplicate detection	`[1, 3, 3, 7]`	`3`
Frequency count	`leave, leave, sick`	`leave=2, sick=1`

14 Java Track

JAVA FUNDAMENTALS

- OOP, classes, interfaces
- Inheritance, polymorphism, encapsulation
- Exceptions
- Collections
- Generics

MODERN JAVA

- Lambda
- Streams
- Optional
- Functional interfaces
- Date/time API

Expected evidence: an `Employee` class with private fields; a collection or stream that filters by department; `Optional` instead of `null`; a spoken difference between an interface and a class.

15 Spring Boot

CONTROLLER SERVICE REPOSITORY JPA / HIBERNATE DATABASE

Spring Boot REST Dependency Injection DTO Entity Validation Exception handling Logging JPA Hibernate Spring Security JWT Testing

```
EmployeeController.create(dto)
  → EmployeeService.create(dto)
      validate name/email
      reject duplicate email
      load department or fail
  → EmployeeRepository.save(entity)
  → 201 + EmployeeResponse
```

Layer	Owns	Must not own
Controller	HTTP status, DTO in/out	Unique-email rule, SQL
Service	Business rules, roles	Servlet / request objects
Repository	Persistence	Status codes
Entity	Table mapping	Password or JWT logic

16 Python Track

Variables · Lists · Tuples · Sets · Dictionaries · Functions · Modules · Packages · Exceptions · File handling · OOP · Type hints · Generators · Decorators · Async

Expected evidence: a typed function with Pytest; grouping records with a dictionary; a Pydantic model or dataclass for an employee; one sentence each for list, tuple, and set.

17 FastAPI

ROUTER · SERVICE · REPOSITORY · SQLALCHEMY · DATABASE

Routes · Request models · Response models · Pydantic · Validation · Dependency injection · Middleware · Exceptions · Authentication · Async · API documentation

```
employees.router.create(payload)
  → EmployeeService.create(payload)
      validate with Pydantic
      reject duplicate email
      load department or fail
  → EmployeeRepository.save(model)
  → 201 + EmployeeOut
```

Java and Python use different class names and the same rules: validation, unique email, role check, and one error JSON shape.

18 Frontend — React + TypeScript

Trainees learn components, props, state, hooks, forms, validation, routing, protected routes, API integration, error handling, loading states, and responsive design.

```
src/
├── components/
├── pages/
├── layouts/
├── services/
├── hooks/
├── types/
├── utils/
├── routes/
└── assets/
```

Data	Lives in	Why
Login form fields	`LoginPage` local state	Only that page edits them
Auth token / current user	Auth context	Many routes need it
Employee list	Page state or hook	Fetched for that screen
API calls	`services/*.ts`	JSX must not contain URLs

Required screens by Day 35: `/login`, `/dashboard`, `/employees` (list + create), shared layout, empty/loading/error states.

SCREEN INVENTORY

Route	Role	Fields / actions	Empty / error copy
`/login`	Public	Email, password, Submit	“Email or password is wrong.”
`/dashboard`	All signed-in	Employees, present today, pending leave	“No data yet. Add employees first.”
`/employees`	ADMIN, HR	Name, email, department, joining date, role, Add	“No employees match this filter.”
`/employees/new`	ADMIN, HR	Name, email, department, joining date, role	Inline field errors
`/employees/:id`	ADMIN, HR; EMPLOYEE own	Same fields; EMPLOYEE cannot change role	“Employee not found.”
`/departments`	ADMIN, HR	Name, employee count, Add	“No departments yet.”
`/attendance`	ADMIN, HR; EMPLOYEE own	Date, employee, Present/Absent/Leave	“No attendance for this date.”
`/leaves`	All	Dates, type, status; Approve/Reject for HR/ADMIN	“No leave requests.”
`/profile`	All	Own name, email, department, password change	Validation on email and password

Shared layout: product name, signed-in user, role badge, logout. Missing token redirects to `/login`.

19 Database

Trainees learn SQL, tables, keys, constraints, relationships, joins, indexes, transactions, normalization, and query optimization.

ORM: Java uses JPA / Hibernate. Python uses SQLAlchemy.

> ORM knowledge must not replace SQL knowledge.

```
departments(id, name UNIQUE)
employees(id, name, email UNIQUE, department_id FK, joining_date, role, password_hash)
attendance(id, employee_id FK, work_date, status, UNIQUE(employee_id, work_date))
leaves(id, employee_id FK, start_date, end_date, type, status)
```

Required by-hand queries: employees with department names (`INNER JOIN`); leave count by department (`LEFT JOIN` + `GROUP BY`); a transaction that approves leave and writes related attendance or rolls both back.

```
BEGIN;
UPDATE leaves SET status = 'APPROVED' WHERE id = 18 AND status = 'PENDING';
-- if row count is 0, ROLLBACK and return 409
INSERT INTO attendance (employee_id, work_date, status)
VALUES (41, '2026-08-20', 'LEAVE');
COMMIT;
```

SEED DATA

Table	Rows
departments	Engineering, Human Resources, Finance
employees	Asha Patil (HR), Kiran Shah (ADMIN), Rohan Deshmukh (EMPLOYEE, Engineering)
attendance	Rohan Present on 2026-08-18 and 2026-08-19
leaves	Rohan 2026-08-20 to 2026-08-21, CASUAL, PENDING

Passwords are hashed. These values are examples, not production credentials.

20 REST API

Both backend tracks implement the same business API so Java and Python trainees learn the same API design principles.

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate user
GET	/api/employees	List employees
POST	/api/employees	Create employee
GET	/api/employees/{id}	Get employee
PUT	/api/employees/{id}	Update employee
DELETE	/api/employees/{id}	Delete employee
GET	/api/departments	List departments
POST	/api/departments	Create department
GET	/api/attendance	List attendance
POST	/api/attendance	Record attendance
GET	/api/leaves	List leaves
POST	/api/leaves	Apply for leave
PUT	/api/leaves/{id}/approve	Approve leave

ENDPOINT NOTE

Document every endpoint the trainee implements. Keep the notes in `docs/api.md` or OpenAPI. Do not leave the contract only in a mentor's memory.

Field	Example
Purpose	Create an employee
Roles	ADMIN, HR
Request	`{ "name", "email", "departmentId", "joiningDate" }`
Success	`201` + employee JSON
Errors	`400` validation, `401` missing token, `403` wrong role, `409` duplicate email

```
{
  "error": "VALIDATION_FAILED",
  "message": "Email is required",
  "field": "email"
}
```

REMAINING CONTRACTS THE TRAINEE MUST WRITE

Endpoint	Rule
`GET /api/employees`	`200` array. Empty is `[]`, not `404`. Filter `?departmentId=3`.
`PUT /api/employees/{id}`	Same roles as create. Unknown id `404`. EMPLOYEE may change own name only.
`DELETE /api/employees/{id}`	ADMIN only, `204`. Pending leave blocks delete with `409`.
`GET/POST /api/departments`	Create requires unique name. Duplicate `409`.
`GET/POST /api/attendance`	Body employeeId, workDate, status. Duplicate date `409`.
`POST /api/leaves`	Dates and type. Employee id from token. Overlap `409`. Starts `PENDING`.
`GET /api/health`	No auth. `{ "status": "ok" }` for Compose and CI.

PART III Project & Engineering

21 Major Training Project

Employee Management System

Roles: ADMIN · HR · EMPLOYEE

Authentication | Dashboard | Employee Management | Department Management | Attendance | Leave | Reports | Profile

Module	Minimum complete behavior
Authentication	Login, logout, invalid-credential handling, token stored safely
Dashboard	Role-aware summary of employees, attendance, and pending leave
Employees	Create, read, update, list, and validate required fields
Departments	Create and list; employee belongs to a department
Attendance	Record and list attendance for a date range
Leave	Apply, list own requests, approve/reject by HR or Admin
Reports	At least one SQL-backed report, such as leave count by department
Profile	View and update the signed-in user's own details

A module is not complete until it meets the Definition of Done, including tests, review, and documentation.

PROJECT README

The Employee Management repository must contain a README that another trainee can follow without asking the author.

1. What this system does
2. Roles (ADMIN / HR / EMPLOYEE)
3. How to run locally
4. How to run tests
5. Environment variables (names only, never secret values)
6. API summary or link to `docs/api.md`
7. Known limits

A repository with only source files and no README is not a complete project.

LEAVE APPROVAL — FULL PATH

1. EMPLOYEE submits `POST /api/leaves`. Status becomes `PENDING`.
2. HR opens the pending list. EMPLOYEE does not see the approve action.
3. HR calls `PUT /api/leaves/{id}/approve` with `{ "decision": "APPROVED" }`.
4. Service checks: caller is HR or ADMIN; leave exists; status is `PENDING`.
5. In one transaction the leave becomes `APPROVED`. Related attendance writes, if required, happen in the same transaction.
6. If the second write fails, both roll back and the leave stays `PENDING`.
7. A second approve returns `409`. Reject uses `"REJECTED"` and must not write attendance.

Actor	Apply	Approve	Can see
EMPLOYEE	Own	No	Own requests
HR	Yes	Yes	Pending / team
ADMIN	Yes	Yes	All

22 Full-Stack Architecture

JAVA IMPLEMENTATION

REACT | REST | SPRING BOOT | SERVICE | JPA / HIBERNATE | MYSQL / POSTGRESQL

PYTHON IMPLEMENTATION

REACT | REST | FASTAPI | SERVICE | SQLALCHEMY | MYSQL / POSTGRESQL

Both implementations must satisfy the same business requirements.

23 Authentication & Security

Authentication | Authorization | Password hashing | JWT | Roles | Permissions | CORS | Input validation

SQL injection awareness | XSS fundamentals | Secrets management | Environment variables | HTTPS | Least privilege

> Never commit passwords, API keys, tokens, private keys, or production credentials to GitHub.

LOGIN SEQUENCE

1. User posts `{ "email", "password" }` to `POST /api/auth/login`.
2. Server verifies the password hash and issues a JWT with `sub` and `role`.
3. Frontend stores the token in memory or an httpOnly cookie — never in Git.
4. Later requests send `Authorization: Bearer <token>`.
5. Missing token → `401`. Wrong role → `403`.

Case	Status	Meaning
No token on `/api/employees`	401	Not authenticated
EMPLOYEE posts `/api/employees`	403	Authenticated, not allowed
HR posts a valid employee	201	Allowed and created
HR posts a duplicate email	409	Allowed, but conflicts

24 Testing

Backend

Java: JUnit, Mockito, Spring Boot tests

Python: Pytest, FastAPI testing, mocking

Frontend

- Component testing
- Form testing
- API-state testing
- Error-state testing

API

- Postman
- Positive and negative tests
- Authentication and authorization
- Edge cases

MINIMUM TEST CATALOG — LEAVE APPROVE

Test	Layer	Must prove
`approve_pending_leave_as_hr`	Service	Status becomes APPROVED
`employee_cannot_approve`	Service	Rule fails before save
`approve_without_token`	API	401
`approve_as_employee`	API	403
`approve_already_decided`	API	409
`approve_unknown_id`	API	404
`create_employee_missing_email`	API	400/422 with field email

A feature with only a happy-path test is not tested.

25 Debugging

BUG REPRODUCE READ ERROR LOGS NETWORK API BACKEND DATABASE ROOT CAUSE FIX RETEST REGRESSION

The trainee should learn to determine which layer is responsible rather than randomly changing code.

Symptom	Check first	Typical cause
Empty list, Network 401	Token header	Token missing, expired, or stored wrong
403 on create employee	Role in JWT	Logged in as EMPLOYEE
500 on duplicate email	Exception handler	Unique constraint not mapped to 409
Route blank after refresh	Protected route	Token read as empty string
API cannot reach DB in Compose	DB host name	Using `localhost` inside the API container
CI red, local green	Workflow commands	Tests not run the same way

26 Docker

FRONTEND CONTAINER BACKEND CONTAINER DATABASE CONTAINER

Learn Dockerfile, image, container, network, volume, environment variables, and Docker Compose.

Service	Name	Port	Role
Frontend	`ems-web`	5173	React app
Backend	`ems-api`	8080	Java or Python API
Database	`ems-db`	3306 / 5432	MySQL or PostgreSQL

The API container talks to `ems-db`, not `localhost`. `docker compose up --build` must reach `GET /api/health`.

27 CI/CD

DEVELOPER GITHUB BUILD LINT UNIT TESTS INTEGRATION TESTS DOCKER BUILD ARTIFACT DEPLOYMENT HEALTH CHECK

Job	Fails when
Lint	Formatting or unused imports break the standard
Unit tests	A service rule is wrong
Integration tests	API + database contract is wrong
Docker build	The image cannot be built from the branch
Health check	The app does not respond

The trainee must make CI fail on purpose once, then fix it, and keep both logs.

28 AWS

IAM EC2 S3 RDS CloudFront Route 53 Security Groups CloudWatch

The trainee should understand the purpose of each service before using it.

Service	Purpose in DTTP language
IAM	Who is allowed to do what in AWS
EC2	A virtual machine that can run the app
S3	Object storage for files and build artifacts
RDS	Managed MySQL or PostgreSQL
CloudFront	CDN in front of static or public content
Route 53	DNS for the application hostname
Security Groups	Network allow/deny rules
CloudWatch	Logs and alarms

29 Project Management

Every trainee project should use a professional work breakdown.

Level	Example
Epic	Employee Management
Features	Authentication, Employees, Departments, Attendance, Leave, Reports
User Story	As an administrator, I want to create an employee so that the employee can be managed in the system.
Task	Create employee POST API.
Subtasks	Entity, DTO, Repository, Service, Controller, Validation, Test

This teaches trainees how professional teams convert requirements into engineering work.

PART IV Assessment & Career

30 Definition of Done

A feature is complete only when all of the following are true:

- [ ] Requirement understood
- [ ] Acceptance criteria satisfied
- [ ] UI implemented
- [ ] API implemented
- [ ] Database completed
- [ ] Validation added
- [ ] Security checked
- [ ] Tests written
- [ ] Tests passed
- [ ] Error cases checked

- [ ] Git commit created
- [ ] Pull Request created
- [ ] Code review completed
- [ ] Review comments resolved
- [ ] Documentation updated
- [ ] Mentor approved

WHAT "DOCUMENTATION UPDATED" MEANS

Change	Update this
How to run or install	Project README
Endpoint, status, or payload	`docs/api.md` or OpenAPI
A non-obvious design choice	Short decision note
The day's work	Daily report with commit / PR link

"Documentation updated" is not a checkbox for later. If the notes are missing, the feature is not done.

31 Mentor System

Each trainee should have **1 Primary Mentor** and, where practical, **1 Technical Reviewer**.

MENTOR KPIS

Trainee progress | Code-review quality | Feedback quality | Skill-gap identification | Project completion | Production-readiness improvement

This prevents mentorship from becoming informal or inconsistent.

WEEKLY MENTOR AGENDA

1. Git activity and pull requests
2. Assignment and project progress
3. One code-review example
4. One debugging or blocker discussion
5. Skill-gap and next-week plan
6. Status: Green / Amber / Red

INDIVIDUAL IMPROVEMENT PLAN

- The skill gap
- Expected evidence of improvement
- Practice work for the next 7 days
- Review date
- Mentor responsible

SAMPLE INDIVIDUAL IMPROVEMENT PLAN

```
Trainee: Rohan Deshmukh
Status: AMBER
Gap: Cannot separate 401 from 403; unique-email rule is still in the controller.
Evidence required by 22 Aug:
  1. Service rejects duplicate email; controller has no SQL.
  2. API tests: 401 / 403 / 409.
  3. Oral: explain both status codes without notes.
Practice: move the rule, write the failing test first, add API tests, oral review.
Mentor: Asha Patil
Review date: 22 Aug
```

If the evidence is missing on the review date, the status stays Amber or becomes Red. The plan is a written agreement.

32 Daily Report

Field	Question
Daily Objective	What was planned?
Completed	What was actually completed?
Learning	What was learned?
Problems	What went wrong?
Investigation	What was checked?
Resolution	How was it fixed?
Git Activity	Commits / PRs
Next Plan	Tomorrow’s objectives

A report with only “worked on employees” is incomplete. Name the file, the endpoint, the error, or the commit.

SAMPLE DAILY REPORT

Field	Example
Daily Objective	Add validated POST /api/employees on the Java track
Completed	Endpoint returns 201; duplicate email returns 409
Learning	Unique constraint belongs in the service and the database
Problems	First attempt returned 500 on duplicate email
Investigation	Read Hibernate exception and mapped it in the exception handler
Resolution	Service throws ConflictException; handler returns 409 JSON
Git Activity	`feat(employees): reject duplicate email` · PR #14
Next Plan	Add GET list with department name join

33 Weekly Review

The mentor reviews technical progress, assignment completion, project progress, code quality, problem solving, communication, discipline, Git activity, blockers, and skill gaps.

WEEKLY REVIEW NOTE

The mentor writes a short note the trainee can act on. Verbal feedback that is not written down does not count as a weekly review.

```
Progress: employee create/list works with auth.
Quality: controller still contains the unique-email rule.
Gap: cannot explain 401 vs 403 without notes.
Next week: move the rule to the service; add 401/403 API tests.
Status: AMBER
```

34 Competency Model

Level	Definition
L0	Not exposed
L1	Understands
L2	Performs with guidance
L3	Performs independently — graduation target for core skills
L4	Can review / explain
L5	Can own

Advanced areas such as architecture, CI/CD, and AWS may initially target **L2**.

35 Performance Status

Green	Amber	Red
On track.	Needs improvement.	At risk.

Amber and Red trainees receive an **Individual Improvement Plan**.

36 Weekly Score

Area	Weight
Technical Learning	15
Practical Work	20
Problem Solving	15
Project Contribution	20
Code Quality	10
Testing	5
Communication	5
Discipline	5
Documentation	5
Total	**100**

Score band	Meaning
85–100	Independent, reviewed, documented, and reusable work
70–84	Correct work with guidance and a few review cycles
55–69	Incomplete, fragile, or poorly explained work
Below 55	Missing fundamentals or professional standards

A high weekly score requires Git history, not only a verbal update.

WHAT THE BANDS LOOK LIKE ON CREATE EMPLOYEE

Band	What the mentor sees
85–100	Layered code, 409 mapped, tests, PR note, trainee explains the rule without slides
70–84	Feature works after two review rounds; tests cover the happy path only
55–69	Create works; duplicate email is 500; trainee cannot name the layer
Below 55	No validation, secrets in Git, or no pull request

37 Final Assessment

Category	Weight
Programming	15%
Frontend	10%
Backend	15%
Database / API	10%
Final Project	20%
Problem Solving	10%
Testing / Debugging	5%
Git / CI/CD	5%
Documentation	5%
Communication / Professionalism	5%
Total	**100%**

WHAT A DEVELOPER READY (85+) DEMO PROVES

1. Logs in as HR and as EMPLOYEE and shows the different menus.
2. Creates an employee and shows the `201` in the Network tab.
3. Repeats the same email and shows `409`.
4. Applies leave as EMPLOYEE, approves as HR, retries and shows `409`.
5. Opens the SQL for leave count by department.
6. Points to the service test that blocks EMPLOYEE approve.
7. Starts the stack with Compose and hits `/api/health`.
8. Names one next step and one thing they still cannot own.

A 70–84 trainee can do most of this with mentor prompts. A 55–69 trainee cannot complete leave approve or cannot explain a status code.

TEN-MINUTE DEMO SCRIPT

Minute	Show
0–1	Problem: HR cannot manage people or leave in one system
1–3	Login as HR. Dashboard counts. Employee list.
3–5	Create employee. Duplicate email. Network statuses.
5–7	EMPLOYEE apply leave. HR approve. Retry 409.
7–8	SQL report. One test name and what it proves.
8–9	Compose / health. What is not done yet.
9–10	Questions

38 Graduation Levels

85–100 · Developer Ready
Eligible for Junior Full-Stack Developer responsibilities.

70–84 · Junior Developer — Supervised
Can contribute with closer mentoring.

55–69 · Extended Training
Requires additional structured training.

Below 55 · Management Review
Formal decision required.

39 Production Readiness Gate

Before graduation, the trainee must demonstrate:

- [ ] Requirement understanding
- [ ] System design
- [ ] React development
- [ ] Backend development
- [ ] SQL
- [ ] REST APIs
- [ ] Authentication
- [ ] Authorization
- [ ] Testing
- [ ] Debugging
- [ ] Git
- [ ] Pull Requests
- [ ] Code review
- [ ] Docker
- [ ] CI/CD understanding
- [ ] Deployment understanding
- [ ] Documentation
- [ ] Technical communication

40 Career Progression

TRAINEE | FOUNDATION TRAINEE | TECHNICAL TRAINEE | PROJECT TRAINEE | JUNIOR FULL-STACK DEVELOPER | SOFTWARE DEVELOPER | SENIOR DEVELOPER | TECHNICAL LEAD

Promotion must be based on **demonstrated capability**, not simply completion of the 90-day period.

41 DTTP Management System

The long-term goal is an internal Trainee Management System.

ORGANIZATION DASHBOARD

- Total trainees
- Active trainees
- Completed
- Extended
- At-risk

INDIVIDUAL RECORD

- Attendance and daily reports
- Assignments and technical scores
- Git activity, PRs, and code reviews
- Project progress and skill matrix
- Mentor rating and final result

PROFILE | BASELINE | ATTENDANCE | DAILY REPORTS | ASSIGNMENTS | WEEKLY REVIEWS | SKILL MATRIX | PROJECT | CODE REVIEWS | FINAL ASSESSMENT | READINESS | GRADUATION

42 Final DTTP Standard

The program should not ask: *"How many technologies did the trainee complete?"*

> What can this trainee independently build, test, debug, explain, document and maintain?

STUDENT | TRAINEE | PRACTITIONER | PROJECT CONTRIBUTOR | INDEPENDENT TRAINEE | PRODUCTION-READY | JUNIOR FULL-STACK DEVELOPER

DESHMUKH TECHNOLOGIES

DTTP — Deshmukh Technologies Trainee Program

Learn. Build. Solve. Review. Deploy. Grow.

A disciplined, technically capable and production-ready Junior Full-Stack Developer.